

CPE 301 Final Project Part 3

Group Members: Nicole Eames, Georgia Guillebeau, Trinity Hayes

1. Project Motivation/Introduction

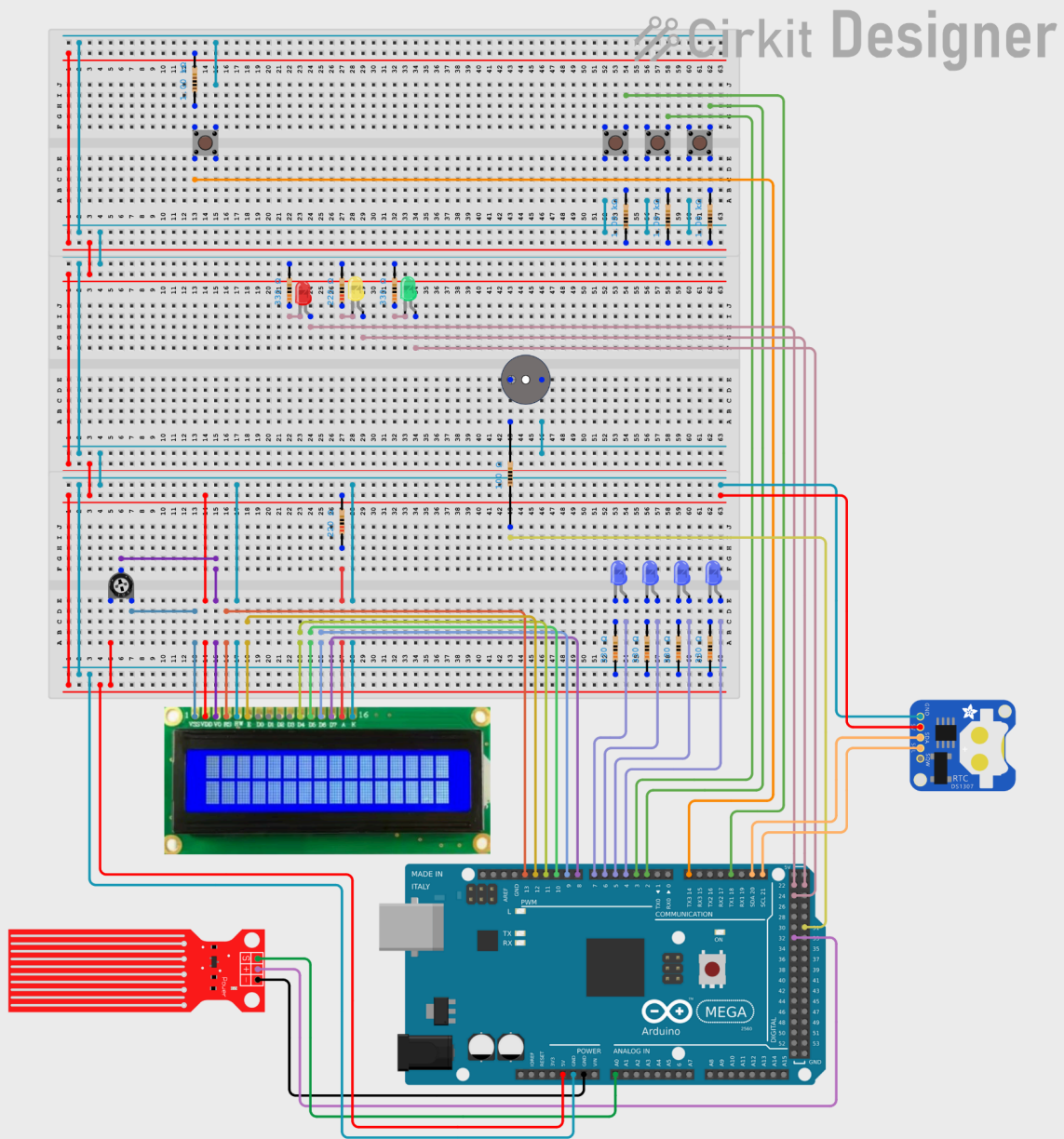
Our project seeks to model a flooded traffic intersection with traffic lights. This intersection is modeled by one traffic light. It also contains a model of inductive loops, so as to simulate outside input. Inductive loops for traffic detection are electromagnetic systems embedded in the pavement that detect vehicle locations. Essentially they are the circles in the ground that tell the traffic light a car is present when a car stops on them.

The purpose of this model is to demonstrate the functionality and behaviour of a traffic-actuated signal in times of flood, and in normal times. That is, a traffic light that changes depending on the input from a button, as opposed to a fixed-time signal, which changes on a clock independent of cars. This project idea came from spending lengthy amounts of time stopped at traffic lights wondering if there was a better way to get cars through the intersection faster. The project has since evolved from this original motivation to better suit the project requirements.

In a much larger scope, an embedded system to control a traffic light is a good design choice, and one that is used quite heavily. Embedded systems are meant to do a specific task continuously with efficiency and reliability. This is exactly what a traffic light requires. In order to create smooth traffic flow, the microcontroller in the traffic light must read input from sensors (if the intersection has them), then reliably switch the lights accordingly. People also have the ability to update the software on the microcontrollers to modify how the light works without changing any hardware in the physical traffic light.

2. Proposed System Overview

The system is meant to simulate a traffic intersection in times of flood. A single traffic light is used for ease of simulation. This traffic light is made of one red, one yellow, and one green LED. This traffic light changes to allow a car to pass if a car is present, and changes to a flashing red light when a flood is detected. The system also plays a sound on a buzzer when it detects that the road is flooded. Doing this, as well as turning the intersection into a four way stop, could help increase caution during hazardous driving conditions. The user can interact with the system by pressing a button to simulate a car, or by simulating a flooded road by dipping a water sensor into a cut of water.



2.2 Give a detailed component list with references

- ATmega 2560 Microcontroller
 - https://drive.google.com/file/d/1WEQ3SOB7dhOXkHlhwrU6zblOQy8fNtmG/view?usp=drive_link
- LCD 1602 Module
 - <https://drive.google.com/file/d/16ELBI1IakaeGQ5XVLIC6TRFJCyyFOI-8/view?usp=sharing>
- Water Level Detection Sensor Module
 - <https://drive.google.com/file/d/1EFc6VsMde43b4nNVXDRgVubfOzBUgJgN/view?usp=sharing>
- DS 1307 RTC Module
 - <https://drive.google.com/file/d/1euyIvcI2GIaaV6su19Yw2C7xn9rVKTd/view?usp=sharing>
- Active Buzzer
 - <https://drive.google.com/file/d/1pmYPDqY7h8qsJhWvoBAmeBp9PrnnZJPW/view?usp=sharing>
- Potentiometer 10K
 - <https://drive.google.com/file/d/1gXa1goG941drNTJvGnEzUjco5EudplEV/view?usp=sharing>
- Momentary Button
 - <https://docs.arduino.cc/libraries/mt-arduino-momentary-button/>
- Full Breadboards
 - <https://drive.google.com/file/d/1oJoFNW6ux1MZyOEWWy3hE8LnNPAWi7Ap/view?usp=sharing>
- Breadboard Jumper Wires
 - https://drive.google.com/file/d/18K-0O_pVbT0JnogOZyhW3JWpYiDovWXw/view?usp=sharing
- Resistors (1k Ω , 330 Ω , 220 Ω , 100 Ω)
 - <https://drive.google.com/file/d/18JujkMwYTUSXifyaa0hq88zs-OOZa01Z/view?usp=sharing>
- LEDs (Red, Yellow, Green, Blue)
 - https://drive.google.com/file/d/1IUekRiP3hW6xkrYcy7Fd07_PyxGN33Y3/view?usp=sharing

3. Expected System Behavior (High-Level)

When the system is initially turned on, it will run its setup loop in code, which will initialize everything it needs to interface with the input and output devices. The system starts in the off state. In this state, all traffic LEDs are off, the LCD is off, and the water sensor is off. The only way to transition the system to the idle state is by pressing the on button. The system also contains a Real Time Clock Module, (RTC), which tracks when the system changes states, and when the actuator is turned on and off. These changes are logged in the serial monitor.

Once the on button is pressed, the system enters idle state. During this state, the traffic light remains red until a car approaches. This approach is emulated by a button push. Once this button is pushed, the traffic light cycles through green, yellow, then red, to simulate the car passing. This can be done as many times as desired while in idle state. While in idle state, the system also collects sensor readings. That is, the water sensor is constantly checking the water level. These water levels are displayed on the LCD, along with the current state. In our program, two threshold water levels were set. The first signifies what water level constitutes a flooded road.

When the system detects a flooded road, the system enters the active state. During the active state, a sound plays on the buzzer to indicate an alarm for drivers. Also, the traffic LEDs no longer function as a standard traffic light. Instead, they flash red to simulate an intersection that behaves as a four way stop. Their behavior is also no longer affected by a car's approach.

The second water level threshold is used to simulate a sensor error. Realistically, this should be an actual error, like when the sensor reads 1023, or reads one value for a long time. However, in the interest of testing the error state functionality, the system transitions to an error state when the water goes over a specific threshold. When the system is in an error state, the traffic LEDs are all off, the buzzer is off, and the LCD displays an error message. The system can be brought back to an idle state by pressing the reset button, or brought back to the off state by pressing the off button.

Each state and actuator transaction is logged in real time to the serial monitor via the RTC. Similarly, each state is denoted by its state LED.

3.1. Module Design

- **Sensor Reading Module**

This module is responsible for acquiring data, both digital and analog, from the outside environment. It is also responsible for triggering interrupts if the predetermined threshold values are achieved. This module also includes communication protocols such as USART, I2C, or SPI that are used to get data from sensors.

- **Water Sensor Submodule**

Responsible for tracking the water level in the environment through the water level detection sensor.

- **Traffic Button Submodule**

Responsible for tracking if the button that corresponds to the car is being pressed. This submodule uses GPIO to communicate, as the hardware it contains consists of buttons, resistors and wires hardware.

- **Display Module**

This module is responsible for providing feedback about the system status to the user. It contains measures to show sensor values, system state, and general system function to the user. It also contains the communication protocols for communication between the microcontroller and the display device. The general display module also contains the LEDs that correspond to system state.

- **LCD Display Submodule**

Responsible for displaying system state and sensor value in words on the LCD.

- **LED Display Submodule**

This submodule includes the LEDs that serve as representations of the traffic light. It is thus responsible for displaying general system functions to the user.

- **Actuator Module**

The actuator module contains the only actuator in the system: the buzzer. In the general design of the system, the buzzer makes sound when the Water Level Detection Sensor passes a predetermined threshold. In terms of modules, this implies that the Actuator Module reacts to the Control Logic Module's interpretations of data from the Water Sensor Submodule.

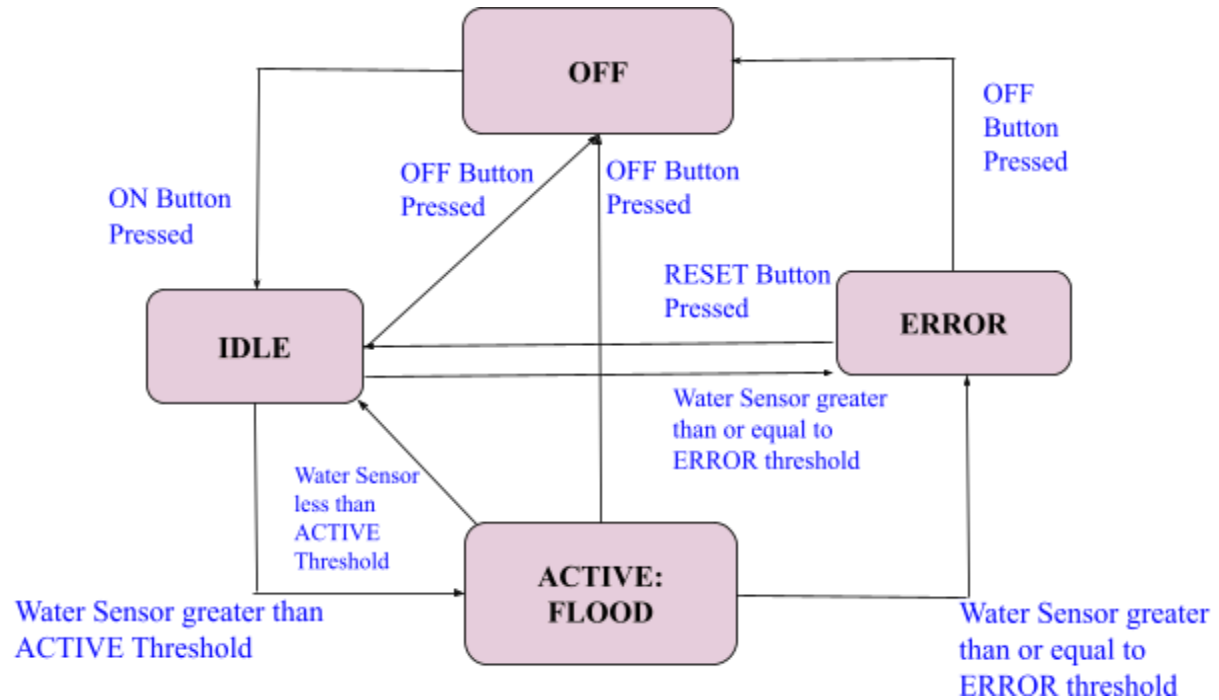
- **Communication (Serial) Module**

This module handles all communication between the microcontroller and the outside system. This means that this module contains all the logic to assure the microcontroller sends and receives data from the serial monitor. This module also contains the RTC, as it is used to create time stamps for every serial output communication.

- **Control Logic Module (Data Processing)**

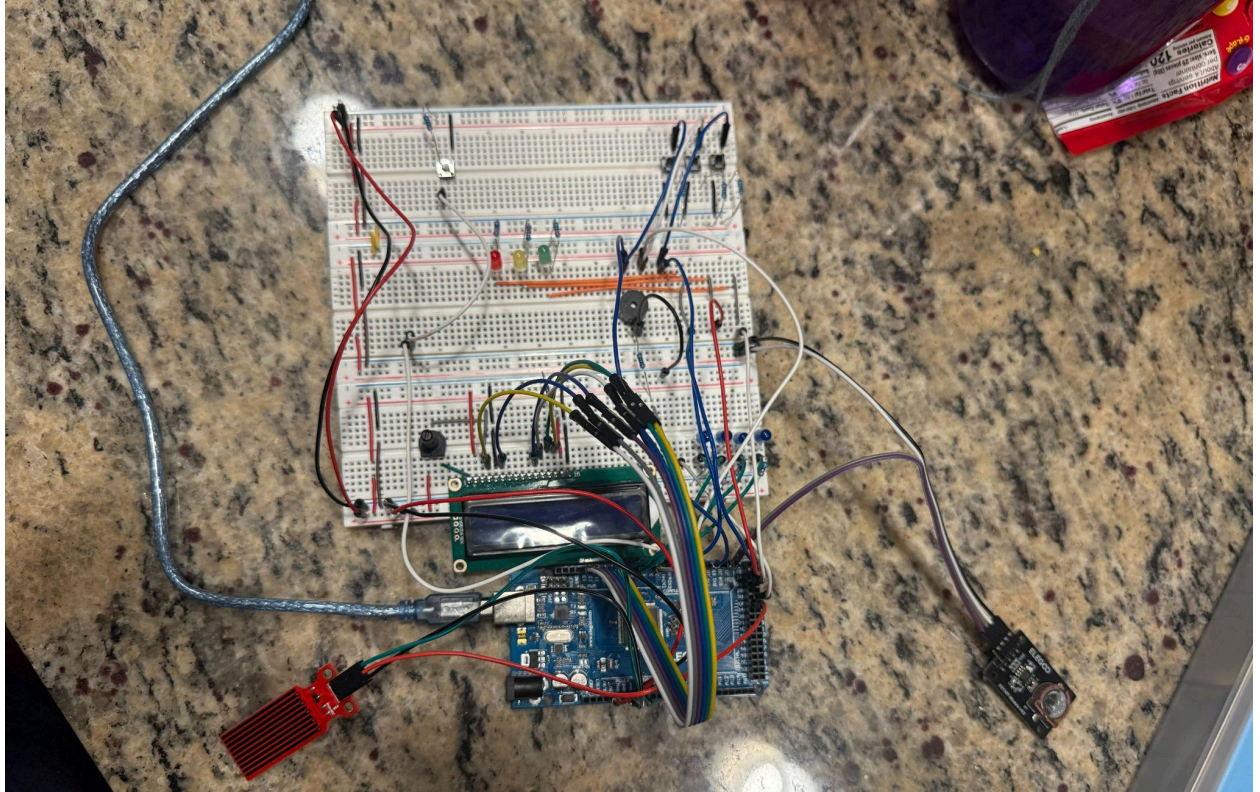
The control logic module runs on the CPU of the microcontroller. It contains all the code that decides the state of the system, and decides when the system transitions states. It makes these decisions by executing algorithms that utilize data from other modules, and by controlling the output of other modules. For instance, if the data that is pulled from the Water Sensor Submodule reflects a value above the predetermined threshold, the Control Logic Module will make actions take place in the Actuator Module, alter the Display Module so as to reflect the state change, and use the Communication Module to communicate this to the user. In this regard, the Control Logic Module connects modules by making decisions from and for them, but does not allow other modules to communicate directly with each other.

3.2 Control Logic (State Diagram or Flowchart)

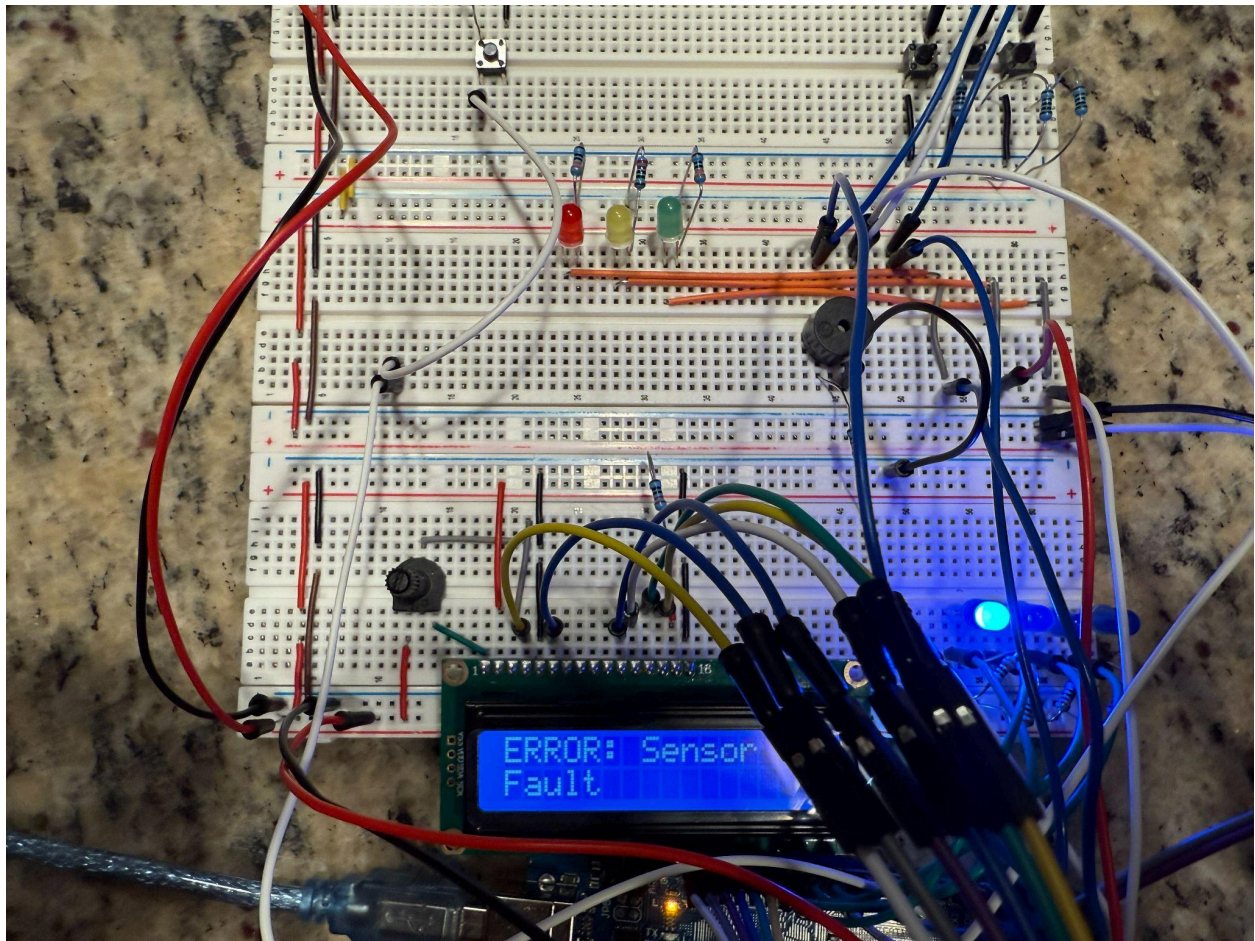


4. Result

Full Circuit Image:



OFF State:



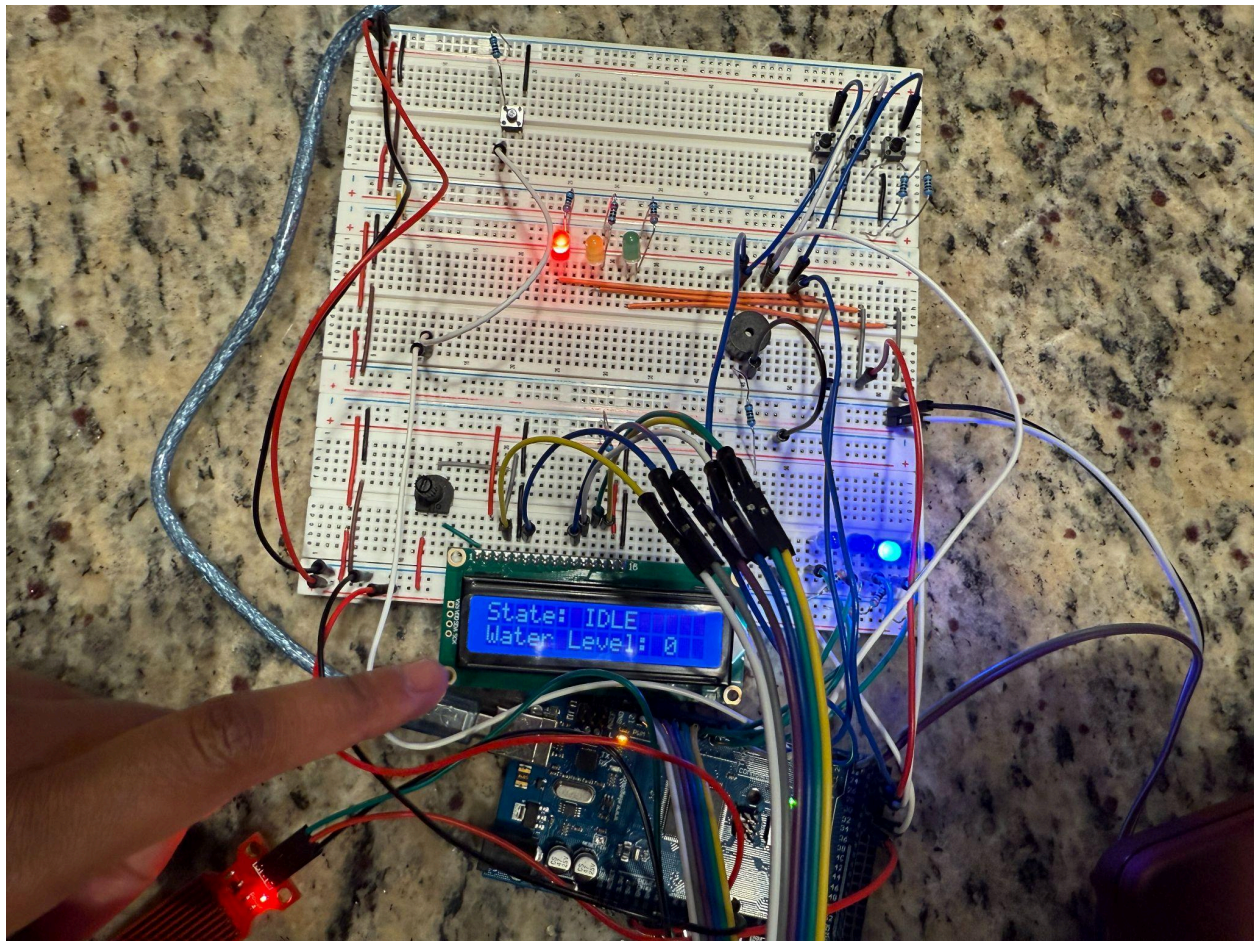
Output Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM6')

No Line Ending 9600 baud

19:42:59 - OFF state entered
19:42:59 - Buzzer OFF

IDLE state:



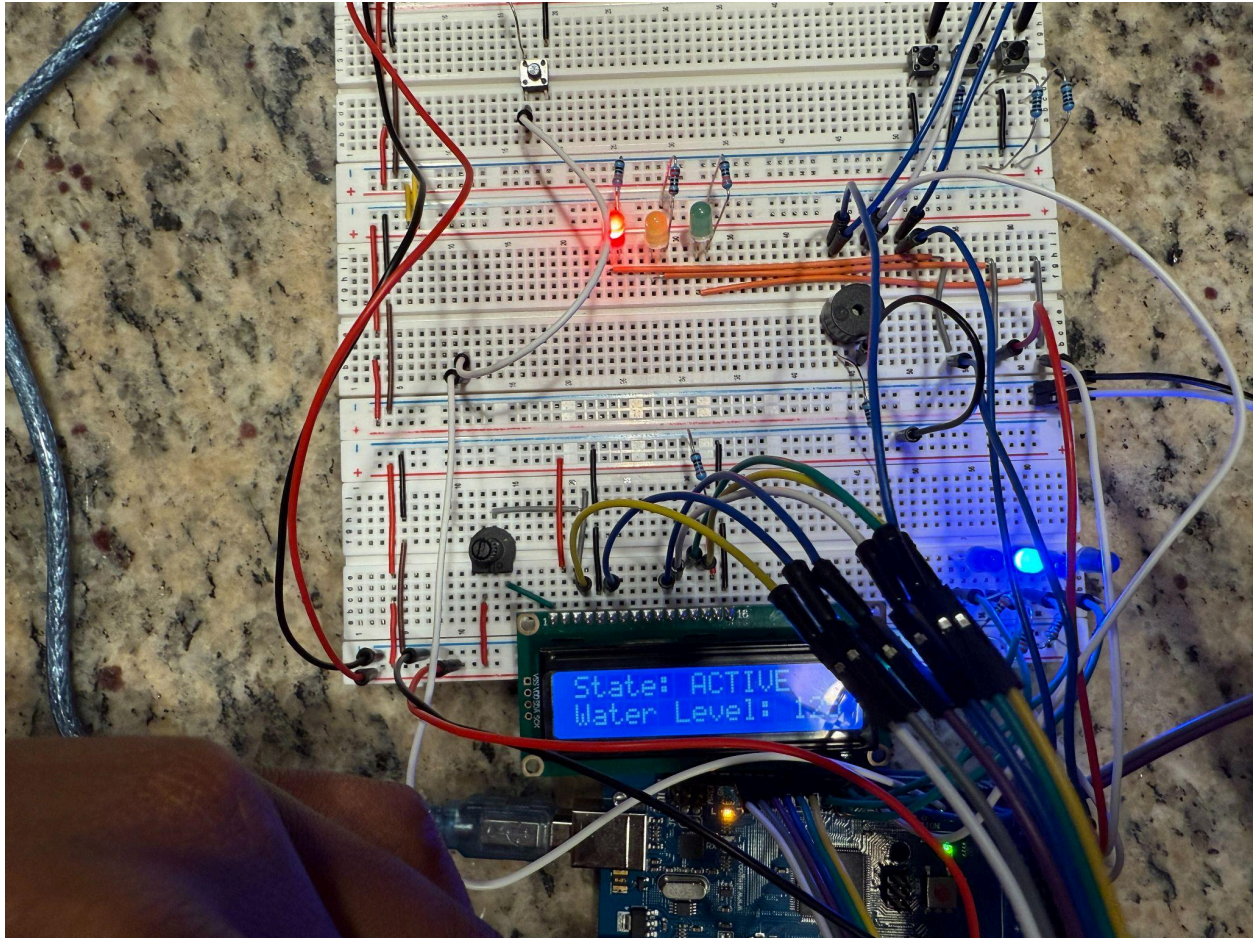
Output Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM6')

No Line Ending 9600 baud

```
19:42:59 - OFF state entered
19:42:59 - Buzzer OFF
19:43:1 - IDLE state entered
19:43:1 - Buzzer OFF
```


ACTIVE state:



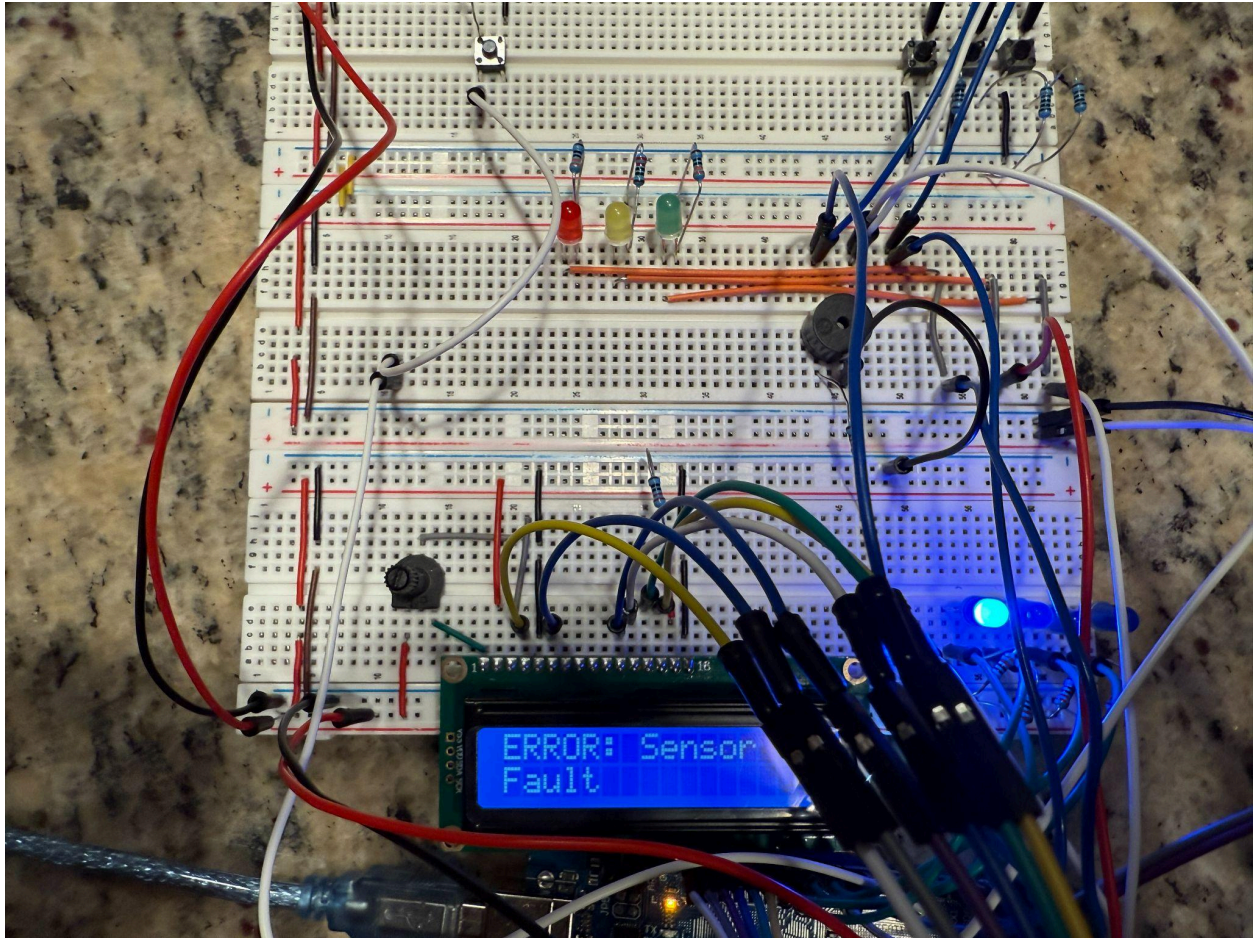
Output Serial Monitor x

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM6')

No Line Ending 9600 baud

```
19:42:59 - OFF state entered
19:42:59 - Buzzer OFF
19:43:1 - IDLE state entered
19:43:1 - Buzzer OFF
19:44:52 - ACTIVE state entered
19:44:52 - Buzzer ON
```

ERROR state:



Output Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM6')

No Line Ending 9600 baud

```
19:42:59 - OFF state entered
19:42:59 - Buzzer OFF
19:43:1 - IDLE state entered
19:43:1 - Buzzer OFF
19:44:52 - ACTIVE state entered
19:44:52 - Buzzer ON
19:45:17 - ERROR state entered
19:45:17 - Buzzer OFF
```


5. Environmental & Energy Impact

- **Energy Efficiency:** The goal of the design is to be able to automate the reaction of a traffic-light during a natural disaster. This would essentially reduce the power utilized by traffic lights by inducing a flashing state (red light) once a flood is detected. This would save power because it will not be making decisions to switch between red, yellow, and green states.
- **Design Safety:** In a large-scale design, we want to ensure a safe transfer of energy transmitted within the system and its interactions with elements in the system. In our simulations we will use pull down resistors and LED resistors to control over excessive deposit of power. We will use a 100 Ohms resistor to power the buzzer and not cause an excess amount of power.
- **Affordability:** Components are cost effective because they come from the course kit. LEDs and buttons are also easy to use and replace. Cost efficiency of the items will allow for the design to be scaled with ease and still ensure reliability since the buttons and water sensor handle input and LED lights and sound buzzer represent responsive output.
- **Sustainability:** Most of the logic will be code, which is easy to update, maintain and track. Code for recognizing input can be easily refined and corrected in real time, meaning the system can continue to be enhanced remotely after the initial build. Additionally the sound buzzer can be swapped easily if there are any desired hardware upgrades, and the water sensor can be replaced should it break or need to be upgraded to detect more efficiently.
- **Accessibility:** Buttons are easy to utilize as a mode of interactive simulation, both to trigger traffic light reactions, and state changes. The project will serve as a proxy of users being at a traffic stop and experiencing an unexpected flood. With the aid of a visual water source, which is automatically detected by the water sensor (we will dunk the sensor). Intuitively this would be automated at a traffic stop and the user would obey as rules regulate at a red blinking 4-way stop, however for the process of demonstration it is scaled down to a singular traffic light.

6. Conclusion

Overall, the scaled down implementation of our system did work. The scaled down version, reduced the number of traffic stops and simplified the amount of states induced during system functionality. The off state properly kept the water sensor, buzzer and traffic LED'S off despite any external attempts to induce the active state (flooding the system). During the idle state, at any time a car was sensed, the traffic light cycled through the green, yellow, red cycle, which held true for consecutive cars passing. During this state, the system appropriately checked for sensor readings that indicated a state change. Once the active state was entered the red LED flashed and the buzzer remained active until the state was exited. During the active state the system appropriately checked for the water threshold to exceed the limit to enter the error state. Finally, once in the error state, the system remained in error even if the water threshold was reduced, until either the system was reset or turned off. The system also fulfilled the requirement that the off state could be enabled from any state.

The biggest limitation of our project is our use of only the kits we have from CPE 301 and CPE 201. This also imposes the cost constraint of spending \$0. With the scaled down implementation we only see how a one-lane traffic stop interacts with all of the states. However, a scaled up eight lane system would also have to account for multiple directions, requiring more interrupts that simulate cars. This greater number of interrupts imposes the constraint posed by the number of digital pins on an ATmega 2560 which is ~53 digital pins. Eight traffic lights means 24 pins for LEDs. One button from left turn lanes means 4 pins for buttons each requiring an interrupt. Therefore, the ATmega 2560 would not support the sufficient number of GPIO pins that are specified for interrupts. Adding sensors for water level detection, LCD display, and an RTC module will take even more pin usage. With the cumulative number of pins required to show full system implementation, its scope is not feasible with a singular ATmega 2560 microcontroller. As far as hardware implementations, the water sensor is limited from being connected to the GPIO pin and powered directly from the microcontroller. Its range decreased significantly, from a maximum of ~400 to ~100, compared to when it is powered directly from the board.

7. Video Demonstration:

Link to the Demonstration Video Google Drive:

<https://drive.google.com/file/d/1g92qAlEibO2YxDJcJEwq0xy98j-ga94h/view?usp=sharing>

8. GitHub Repository Link

Link to the Github Repository:

<https://github.com/nicole-eames/CPE-301-Final-Project.git>